Embedded Systems Lab-05: Programs on Sorting

1) Write an ALP to sort an array using selection sort

```
    AREA    RESET, DATA, READONLY
    EXPORT __Vectors
__Vectors
        DCD 0X10001000
        DCD Reset_Handler
        ALIGN 4
        AREA    mycode, CODE, READONLY
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
    BL MEMINIT          ; Function to initialize R/W mem. with array.
    LDR R0, =SRC        ; Get base index of array
    MOV R1, #0          ; Outer loop offset
OUTER
    MOV R2, R1          ; Minimum element index
    ADD R3, R1, #4      ; Inner loop offset
INNER
    LDR R4, [R0, R2]    ; Get min. ele.
    LDR R5, [R0, R3]    ; Get current ele.
    CMP R5, R4
    MOVLO R2, R3        ; If current ele. < min. ele., rewrite min. index
    ADD R3, #4          ; Update inner loop offset
    CMP R3, #40
    BNE INNER           ; if R3 != 40, continue INNER
```

```
    LDR R6, [R0, R2]
    LDR R7, [R0, R1]
    STR R6, [R0, R1]
    STR R7, [R0, R2]    ; Swap current outer loop ele. and discovered min. ele.
    ADD R1, #4          ; Update outer loop offset
    CMP R1, #36
    BNE OUTER           ; if R1 != 36, continue
STOP
    B STOP
MEMINIT
    LDR R0, =PRG        ; Load array base address in RO area
    LDR R3, =SRC        ; Load array base address in RW area
    MOV R1, #0          ; Loop offset
LOOP
    LDR R2, [R0, R1]
    STR R2, [R3, R1]    ; Transfer from PRG to SRC
    ADD R1, #4          ; Update loop offset
    CMP R1, #40
    BNE LOOP            ; If R1 != 40, continue
    BX LR               ; Return
    ALIGN
PRG DCD 2,1,7,6,8,0,9,5,4,3
    AREA mydata, DATA, READWRITE
SRC DCD 0
    END
```

Register	Value
Core	
R0	0x10000000
R1	0x00000024
R2	0x00000020
R3	0x00000028
R4	0x00000008
R5	0x00000009
R6	0x00000008
R7	0x00000008
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x10001000
R14 (LR)	0x0000000D
R15 (PC)	0x0000003A
xPSR	0x61000000

Memory 1

Address: 0x10000000

```
0x10000000: 00 00 00 00 01 00 00 00 02 00 00 00 03 00 00 00 04 00 00 00 05 00 00
0x10000017: 00 06 00 00 00 07 00 00 00 08 00 00 00 09 00 00 00 00 00 00 00 00 00
0x1000002E: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

It can be seen that the array is correctly sorted and stored in the RW data memory area.

2) Write an ALP to find factorial of an unsigned number using recursion

```
    AREA    RESET, DATA, READONLY
    EXPORT __Vectors
__Vectors
        DCD 0X10001000
        DCD Reset_Handler
        ALIGN 4
        AREA    mycode, CODE, READONLY
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
    MOV R0, #5          ; Input value
    BL FACTORIAL        ; Function call
STOP
    B STOP
FACTORIAL
    CMP R0, #0          ; Base case R0 = 0
    MOVEQ R0, #1        ; 0! = 1
    BXEQ LR             ; Return from base case
    STMDB SP!, {R0, LR} ; Push current input and link register value onto stack
    SUB R0, R0, #1      ; Update input for next function call to R0 - 1
    BL FACTORIAL        ; Recursive function call
    LDMIA SP!, {R1, LR} ; Pop previous input and link register values from stack
    MUL R0, R1, R0      ; Calculate factorial with stack values
    BX LR               ; Return
    END
```

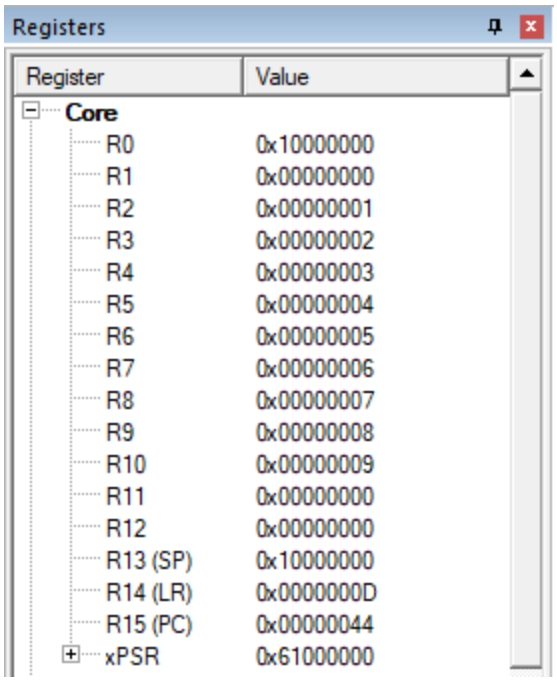
Register	Value
Core	
R0	0x00000078
R1	0x00000005
R2	0x00000000
R3	0x00000000
R4	0x00000000
R5	0x00000000
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x10001000
R14 (LR)	0x00000011
R15 (PC)	0x00000010
xPSR	0x61000000

We observe that the correct value of 5!, 120, is stored in R0 (78 in base 16 is 120 in base 10).

3) Assume that 10 32-bit numbers are stored in registers R1-R10. Sort these numbers in the empty ascending stack using selection sort and store the sorted array back into the registers. Use STM and LDMDB instructions wherever necessary.

```
    AREA    RESET, DATA, READONLY
    EXPORT __Vectors
__Vectors
        DCD 0X10001000
        DCD Reset_Handler
        ALIGN 4
        AREA    mycode, CODE, READONLY
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
    BL REGLOADER
    LDR R0, =SRC          ; Gets base index of stack
    MOV SP, R0            ; Sets stack pointer to base index
    STMIA SP!, {R1-R10} ; Pushes all register contents into stack
    MOV R1, #0            ; Outer loop offset
OUTER
    MOV R2, R1            ; Minimum element index
    ADD R3, R1, #4        ; Inner loop offset
INNER
    LDR R4, [R0, R2]      ; Get min. ele.
    LDR R5, [R0, R3]      ; Get current ele.
    CMP R5, R4
    MOVLO R2, R3          ; If current ele. < min. ele., rewrite min. index
    ADD R3, #4            ; Update inner loop offset
    CMP R3, #40
    BNE INNER             ; if R3 != 40, continue INNER
```

```
    LDR R6, [R0, R2]
    LDR R7, [R0, R1]
    STR R6, [R0, R1]
    STR R7, [R0, R2]      ; Swap current outer loop ele. and discovered min. ele.
    ADD R1, #4            ; Update outer loop offset
    CMP R1, #36
    BNE OUTER             ; if R1 != 36, continue
    LDMDB SP!, {R1-R10} ; Loads values back into registers
STOP
    B STOP
REGLOADER                 ; initializes R1-R10 with values
    MOV R1, #2
    MOV R2, #4
    MOV R3, #7
    MOV R4, #1
    MOV R5, #3
    MOV R6, #9
    MOV R7, #0
    MOV R8, #6
    MOV R9, #5
    MOV R10, #8
    BX LR
    ALIGN
    AREA mydata, DATA, READWRITE
SRC DCD 0
    END
```



Registers

Register	Value
Core	
R0	0x10000000
R1	0x00000000
R2	0x00000001
R3	0x00000002
R4	0x00000003
R5	0x00000004
R6	0x00000005
R7	0x00000006
R8	0x00000007
R9	0x00000008
R10	0x00000009
R11	0x00000000
R12	0x00000000
R13 (SP)	0x10000000
R14 (LR)	0x0000000D
R15 (PC)	0x00000044
xPSR	0x61000000

It can be seen that registers R1-R10 hold the values in non-descending order.

4) Repeat question 4 for a fully descending stack using STMDB and LDM instruction wherever necessary

```
    AREA    RESET, DATA, READONLY
    EXPORT __Vectors
__Vectors
        DCD 0X10001000
        DCD Reset_Handler
        ALIGN 4
        AREA    mycode, CODE, READONLY
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
    BL REGLOADER
    STMDB SP!, {R1-R10} ; Pushes all register contents into stack
    MOV R0, SP          ; Stores base index of stack
    MOV R1, #0          ; Outer loop offset
OUTER
    MOV R2, R1          ; Minimum element index
    ADD R3, R1, #4      ; Inner loop offset
INNER
    LDR R4, [R0, R2]    ; Get min. ele.
    LDR R5, [R0, R3]    ; Get current ele.
    CMP R5, R4
    MOVLO R2, R3        ; If current ele. < min. ele., rewrite min. index
    ADD R3, #4          ; Update inner loop offset
    CMP R3, #40
    BNE INNER           ; if R3 != 40, continue INNER
```

```
    LDR R6, [R0, R2]
    LDR R7, [R0, R1]
    STR R6, [R0, R1]
    STR R7, [R0, R2]     ; Swap current outer loop ele. and discovered min. ele.
    ADD R1, #4           ; Update outer loop offset
    CMP R1, #36
    BNE OUTER            ; if R1 != 36, continue
    LDMIA SP!, {R1-R10} ; pops stack ele. back to registers
STOP
    B STOP
REGLOADER                ; initializes R1-R10 with values
    MOV R1, #2
    MOV R2, #4
    MOV R3, #7
    MOV R4, #1
    MOV R5, #3
    MOV R6, #9
    MOV R7, #0
    MOV R8, #6
    MOV R9, #5
    MOV R10, #8
    BX LR
    ALIGN
    AREA mydata, DATA, READWRITE
SRC DCD 0
    END
```

Registers

Register	Value
Core	
R0	0x10000FD8
R1	0x00000000
R2	0x00000001
R3	0x00000002
R4	0x00000003
R5	0x00000004
R6	0x00000005
R7	0x00000006
R8	0x00000007
R9	0x00000008
R10	0x00000009
R11	0x00000000
R12	0x00000000
R13 (SP)	0x10001000
R14 (LR)	0x0000000D
R15 (PC)	0x00000042
xPSR	0x61000000

5) Write an ARM ALP that contains a list of numbers and makes a count of a) even and odd numbers, b) numbers greater than 10

```
    AREA    RESET, DATA, READONLY
    EXPORT __Vectors
__Vectors
        DCD 0X10001000
        DCD Reset_Handler
        ALIGN 4
        AREA    mycode, CODE, READONLY
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
    LDR R0, =SRC        ; Load list base address
    MOV R1, #0          ; Even count
    MOV R2, #0          ; Odd count
    MOV R3, #0          ; >10 count
    MOV R4, #0          ; Index offset
LOOP
    LDR R5, [R0, R4]    ; Load current element
    CMP R5, #10
    ADDHI R3, #1        ; If R5 > 10, increment R3
    MOVS R5, R5, LSR #1 ; Check LSB for even/odd
    ADDCC R1, #1        ; If LSB = 0, increment R1
    ADDCS R2, #1        ; else increment R2
    ADD R4, #4          ; Update offset
    CMP R4, #40
    BNE LOOP            ; If offset != 40, continue
STOP
    B STOP
    ALIGN
SRC DCD 2,1,17,6,8,0,9,5,4,3
    END
```

Register	Value
Core	
R0	0x00000034
R1	0x00000005
R2	0x00000005
R3	0x00000001
R4	0x00000028
R5	0x00000001
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x10001000
R14 (LR)	0xFFFFFFFF
R15 (PC)	0x00000032
xPSR	0x61000000

6) Implement full ascending and empty descending stack using LDR, STR, ADD and SUB instructions

```
    AREA    RESET, DATA, READONLY
    EXPORT __Vectors
__Vectors
        DCD 0X10001000
        DCD Reset_Handler
        ALIGN 4
        AREA    mycode, CODE, READONLY
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
    LDR R0, =NUM        ; Numbers to push/pop
    LDR R1, =FAS        ; Full Ascending stack top pointer
    LDR R2, =EDS+20     ; Empty descending stack top pointer
    MOV R3, #0          ; Offset variable
PUSHLOOP
    LDR R4, [R0, R3]    ; Fetch number
    ADD R1, #4          ; Increment FA stack pointer
    STR R4, [R1]        ; Store to FA stack
    STR R4, [R2]        ; Store to ED stack
    SUB R2, R2, #4      ; Decrement ED stack pointer
    ADD R3, #4          ; Update offset
    CMP R3, #20
    BNE PUSHLOOP        ; If offset != 20, continue
```

```
    LDR R5, =OFA          ; Pop storage for FA stack
    LDR R6, =OED          ; Pop storage for ED stack
    MOV R3, #0            ; Reset offset variable
POPLOOP
    LDR R0, [R1]          ; Load top element from FA stack
    SUB R1, R1, #4        ; Decrement FA stack pointer
    STR R0, [R5, R3]      ; Store FA top
    ADD R2, #4            ; Increment ED stack pointer
    LDR R0, [R2]          ; Load top element from ED stack
    STR R0, [R6, R3]      ; Store ED top
    ADD R3, #4            ; Update offset
    CMP R3, #20
    BNE POPLOOP           ; If offset != 20, continue
STOP
    B STOP
    ALIGN
NUM DCD 1, 2, 3, 4, 5
    AREA mydata, DATA, READWRITE
FAS DCD 0, 0, 0, 0, 0, 0
EDS DCD 0, 0, 0, 0, 0, 0
OFA DCD 0, 0, 0, 0, 0
OED DCD 0, 0, 0, 0, 0
    END
```

Registers

Register	Value
Core	
R0	0x00000001
R1	0x10000000
R2	0x1000002C
R3	0x00000014
R4	0x00000005
R5	0x10000030
R6	0x10000044
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x10001000
R14 (LR)	0xFFFFFFFF
R15 (PC)	0x00000048
xPSR	0x61000000

Memory 1

Address: 0x10000000

```
0x10000000: 00 00 00 00 01 00 00 00 02 00 00 00 03 00 00 00 04 00 00 00 05 00 00
0x10000017: 00 00 00 00 00 05 00 00 00 04 00 00 00 03 00 00 00 02 00 00 00 01 00
0x1000002E: 00 00 05 00 00 00 04 00 00 00 03 00 00 00 02 00 00 00 01 00 00 00 05
0x10000045: 00 00 00 04 00 00 00 03 00 00 00 02 00 00 00 01 00 00 00 00 00 00 00
0x1000005C: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

The stack contents and the pop order can be seen correctly stored in memory.